PKG 03: ARRAY BAG

Our bag is an abstract bag. This abstract data type (ADT) bag holds unordered and possibly duplicated objects. The objects have the same or related data types…

```
/**
 * An interface that describes the operations of a bag of objects.
 *
 * @author Frank M. Carrano
 * @author Timothy M. Henry
 * @version 4.1
 * @param <T>
 */
public interface BagInterface<T> {
```

Generic (ADT)

```
    /**
     * Gets the current number of entries in this bag.
     * @return The integer number of entries currently in the bag.
     */
    public int getCurrentSize();

    /**
     * Sees whether this bag is empty.
     * @return True if the bag is empty, or false if not.
     */
    public boolean isEmpty();

    /**
     * Adds a new entry to this bag.
     * @param newEntry The object to be added as a new entry.
     * @return True if the addition is successful, or false if not.
     */
    public boolean add(T newEntry);

    /**
     * Removes one unspecified entry from this bag, if possible.
     * @return Either the removed entry, if the removal. was successful, or
     * null.
     */
    public T remove();

    /**
     * Removes one occurrence of a given entry from this bag.
     * @param anEntry The entry to be removed.
     * @return True if the removal was successful, or false if not.
     */
    public boolean remove(T anEntry);

    /**
     * Removes all entries from this bag.
     */
    public void clear();

    /**
     * Counts the number of times a given entry appears in this bag.
     * @param anEntry The entry to be counted.
     * @return The number of times anEntry appears in the bag.
     */
    public int getFrequencyOf(T anEntry);
```

```
    /**
     * Tests whether this bag contains a given entry.
     * @param anEntry The entry to locate.
     * @return True if the bag contains anEntry, or false if not.
     */
    public boolean contains(T anEntry);

    /**
     * Retrieves all entries that are in this bag.
     * @return A newly allocated array of all the entries in the bag. Note: If
     * the bag is empty, the returned array is empty.
     */
    public T[] toArray();
//    public <T> T[] toArray();  // Alternate
//    public Object[] toArray(); // Alternate

} // end BagInterface
```

```
import java.util.Arrays;

/**
 * A class that implements a bag of objects by using an array. The bag is never
 * full.
 */
public final class ResizableArrayBag<T> implements BagInterface<T> {

    private T[] bag; // Cannot be final due to doubling
    private int numberOfEntries;
    private boolean initialized = false;
    private static final int DEFAULT_CAPACITY = 25; // Initial capacity of bag
    private static final int MAX_CAPACITY = 10000;

    /**
     * Creates an empty bag whose initial capacity is 25.
     */
    public ResizableArrayBag() {
        this(DEFAULT_CAPACITY);
    } // end default constructor

    /**
     * Creates an empty bag having a given initial capacity.
     * @param initialCapacity The integer capacity desired.
     */
    public ResizableArrayBag(int initialCapacity) {
        checkCapacity(initialCapacity);

        // The cast is safe because the new array contains null entries
        @SuppressWarnings("unchecked")
        T[] tempBag = (T[]) new Object[initialCapacity]; // Unchecked cast
        bag = tempBag;
        numberOfEntries = 0;
        initialized = true;
    } // end constructor
```

```
127         /**
128          * Creates a bag containing given entries.
129          * @param contents An array of objects.
130          */
131         public ResizableArrayBag(T[] contents) {
132             checkCapacity(contents.length);
133             bag = Arrays.copyOf(contents, contents.length);
134             numberOfEntries = contents.length;
135             initialized = true;
136         } // end constructor
137
138         /**
139          * Adds a new entry to this bag.
140          * @param newEntry The object to be added as a new entry.
141          * @return True.
142          */
143         @Override
144         public boolean add(T newEntry) {
145             checkInitialization();
146             if (isArrayFull()) {
147                 doubleCapacity();
148             } // end if
149
150             bag[numberOfEntries] = newEntry;
151             numberOfEntries++;
152
153             return true;
154         } // end add
155
156         /**
157          * Retrieves all entries that are in this bag.
158          * @return A newly allocated array of all the entries in this bag.
159          */
160         @Override
161         public T[] toArray() {
162             checkInitialization();
163
164             // The cast is safe because the new array contains null entries.
165             @SuppressWarnings("unchecked")
166             T[] result = (T[]) new Object[numberOfEntries]; // Unchecked cast
167             System.arraycopy(bag, 0, result, 0, numberOfEntries); // end for
168                                 source , start ,   target ,     start    , how many?
169             return result;
170         } // end toArray
171
172         /**
173          * Sees whether this bag is empty.
174          * @return True if this bag is empty, or false if not.
175          */
176         @Override
177         public boolean isEmpty() {
178             return numberOfEntries == 0;
179         } // end isEmpty
180
181         /**
182          * Gets the current number of entries in this bag.
183          * @return The integer number of entries currently in this bag.
184          */
185         @Override
186         public int getCurrentSize() {
187             return numberOfEntries;
188         } // end getCurrentSize
189
```

```
190         /**
191          * Counts the number of times a given entry appears in this bag.
192          * @param anEntry The entry to be counted.
193          * @return The number of times anEntry appears in this ba.
194          */
195         @Override
196         public int getFrequencyOf(T anEntry) {
197             checkInitialization();
198             int counter = 0;
199 
200             for (int index = 0; index < numberOfEntries; index++) {
201                 if (anEntry.equals(bag[index])) {
202                     counter++;
203                 } // end if
204             } // end for
205 
206             return counter;
207         } // end getFrequencyOf
208 
209         /**
210          * Tests whether this bag contains a given entry.
211          * @param anEntry The entry to locate.
212          * @return True if this bag contains anEntry, or false otherwise.
213          */
214         @Override
215         public boolean contains(T anEntry) {
216             checkInitialization();
217             return getIndexOf(anEntry) > -1; // or >= 0
218         } // end contains
219 
220         /**
221          * Removes all entries from this bag.
222          */
223         @Override
224         public void clear() {
225             while (!isEmpty()) {
226                 remove();
227             }
228         } // end clear
229 
230         /**
231          * Removes one unspecified entry from this bag, if possible.
232          * @return Either the removed entry, if the removal was successful, or null.
233          */
234         @Override
235         public T remove() {
236             checkInitialization();
237             T result = removeEntry(numberOfEntries - 1);
238             return result;
239         } // end remove
240 
241         /**
242          * Removes one occurrence of a given entry from this bag.
243          * @param anEntry The entry to be removed.
244          * @return True if the removal was successful, or false if not.
245          */
246         @Override
247         public boolean remove(T anEntry) {
248             checkInitialization();
249             int index = getIndexOf(anEntry);
250             T result = removeEntry(index);
251             return anEntry.equals(result);
252         } // end remove
```

```java
    // Locates a given entry within the array bag.
    // Returns the index of the entry, if located,
    // or -1 otherwise.
    // Precondition: checkInitialization has been called.
    private int getIndexOf(T anEntry) {
        int where = -1;
        boolean found = false;
        int index = 0;

        while (!found && (index < numberOfEntries)) {
            if (anEntry.equals(bag[index])) {
                found = true;
                where = index;
            } // end if
            index++;
        } // end while

        // Assertion: If where > -1, anEntry is in the array bag, and it
        // equals bag[where]; otherwise, anEntry is not in the array.
        return where;
    } // end getIndexOf

    // Removes and returns the entry at a given index within the array.
    // If no such entry exists, returns null.
    // Precondition: 0 <= givenIndex < numberOfEntries.
    // Precondition: checkInitialization has been called.
    private T removeEntry(int givenIndex) {
        T result = null;

        if (!isEmpty() && (givenIndex >= 0)) {
            result = bag[givenIndex];          // Entry to remove
            int lastIndex = numberOfEntries - 1;
            bag[givenIndex] = bag[lastIndex];  // Replace entry to remove with last
entry
            bag[lastIndex] = null;             // Remove reference to last entry
            numberOfEntries--;
        } // end if

        return result;
    } // end removeEntry

    // Returns true if the array bag is full, or false if not.
    private boolean isArrayFull() {
        return numberOfEntries >= bag.length;
    } // end isArrayFull

    // Doubles the size of the array bag.
    // Precondition: checkInitialization has been called.
    private void doubleCapacity() {
        int newLength = 2 * bag.length;
        checkCapacity(newLength);
        bag = Arrays.copyOf(bag, newLength);
    } // end doubleCapacity

    // Throws an exception if the client requests a capacity that is too large.
    private void checkCapacity(int capacity) {
        if (capacity > MAX_CAPACITY) {
            throw new IllegalStateException("Attempt to create a bag whose capacity
exceeds "
                    + "allowed maximum of " + MAX_CAPACITY);
        }
    } // end checkCapacity

```

```
    // Throws an exception if receiving object is not initialized.
    private void checkInitialization() {
        if (!initialized) {
            throw new SecurityException("Uninitialized object used "
                    + "to call an ArrayBag method.");
        }
    } // end checkInitialization
} // end ResizableArrayBag
```

```
/**
 * A demonstration of the class ResizableArrayBag
 */
public class ResizableArrayBagDemo {

    public static void main(String[] args) {
        // A bag whose initial capacity is small
        BagInterface<String> aBag = new ResizableArrayBag<>(3);
        testIsEmpty(aBag, true);

        System.out.println("Adding to the bag more strings than its initial
capacity.");
        String[] contentsOfBag = {"A", "D", "B", "A", "C", "A", "D"};
        testAdd(aBag, contentsOfBag);

        testIsEmpty(aBag, false);
        String[] testStrings2 = {"A", "B", "C", "D", "Z"};
        testFrequency(aBag, testStrings2);
        testContains(aBag, testStrings2);

        // Removing strings
        String[] testStrings3 = {"", "B", "A", "C", "Z"};
        testRemove(aBag, testStrings3);

        System.out.println("\nClearing the bag:");
        aBag.clear();
        testIsEmpty(aBag, true);
        displayBag(aBag);
    } // end main

    // Tests the method add.
    private static void testAdd(BagInterface<String> aBag, String[] content) {
        System.out.print("Adding to the bag: ");
        for (String content1 : content) {
            aBag.add(content1);
            System.out.print(content1 + " ");
        } // end for

        /*
        for (int index = 0; index < content.length; index++) {
            aBag.add(content[index]);
            System.out.print(content[index] + " ");
        } // end for
        */

        System.out.println();

        displayBag(aBag);
    } // end testAdd
```

```
379     // Tests the two remove methods.
380     private static void testRemove(BagInterface<String> aBag, String[] tests) {
381         for (int index = 0; index < tests.length; index++) {
382             String aString = tests[index];
383             if (aString.equals("") || (aString == null)) {
384                 // Test remove()
385                 System.out.println("\nRemoving a string from the bag:");
386                 String removedString = aBag.remove();
387                 System.out.println("remove() returns " + removedString);
388             } else {
389                 // Test remove(aString)
390                 System.out.println("\nRemoving \"" + aString + "\" from the bag:");
391                 boolean result = aBag.remove(aString);
392                 System.out.println("remove(\"" + aString + "\") returns " + result);
393             } // end if
394
395             displayBag(aBag);
396         } // end for
397     } // end testRemove
398
399     // Tests the method isEmpty.
400     // correctResult indicates what isEmpty should return.
401     private static void testIsEmpty(BagInterface<String> aBag, boolean correctResult)
402 {
403         System.out.print("Testing isEmpty with ");
404         if (correctResult) {
405             System.out.println("an empty bag:");
406         } else {
407             System.out.println("a bag that is not empty:");
408         }
409
410         System.out.print("isEmpty finds the bag ");
411         if (correctResult && aBag.isEmpty()) {
412             System.out.println("empty: OK.");
413         } else if (correctResult) {
414             System.out.println("not empty, but it is empty: ERROR.");
415         } else if (!correctResult && aBag.isEmpty()) {
416             System.out.println("empty, but it is not empty: ERROR.");
417         } else {
418             System.out.println("not empty: OK.");
419         }
420         System.out.println();
421     } // end testIsEmpty
422
423     // Tests the method getFrequencyOf.
424     private static void testFrequency(BagInterface<String> aBag, String[] tests) {
425         System.out.println("\nTesting the method getFrequencyOf:");
426         for (int index = 0; index < tests.length; index++) {
427             System.out.println("In this bag, the count of " + tests[index]
428                     + " is " + aBag.getFrequencyOf(tests[index]));
429         }
430     } // end testFrequency
431
432     // Tests the method contains.
433     private static void testContains(BagInterface<String> aBag, String[] tests) {
434         System.out.println("\nTesting the method contains:");
435         for (int index = 0; index < tests.length; index++) {
436             System.out.println("Does this bag contain " + tests[index]
437                     + "? " + aBag.contains(tests[index]));
438         }
439     } // end testContains
440
441
```

```
    // Tests the method toArray while displaying the bag.
    private static void displayBag(BagInterface<String> aBag) {
        System.out.println("The bag contains " + aBag.getCurrentSize()
                + " string(s), as follows:");
        Object[] bagArray = aBag.toArray();
        for (int index = 0; index < bagArray.length; index++) {
            System.out.print(bagArray[index] + " ");
        } // end for

        System.out.println();
    } // end displayBag
} // end ResizableArrayBagDemo
```

```
Testing isEmpty with an empty bag:
isEmpty finds the bag empty: OK.

Adding to the bag more strings than its initial capacity.
Adding to the bag: A D B A C A D
The bag contains 7 string(s), as follows: A D B A C A D
Testing isEmpty with a bag that is not empty:
isEmpty finds the bag not empty: OK.

Testing the method getFrequencyOf:
In this bag, the count of A is 3
In this bag, the count of B is 1
In this bag, the count of C is 1
In this bag, the count of D is 2
In this bag, the count of Z is 0

Testing the method contains:
Does this bag contain A? true
Does this bag contain B? true
Does this bag contain C? true
Does this bag contain D? true
Does this bag contain Z? false

Removing a string from the bag:
remove() returns D
The bag contains 6 string(s), as follows: A D B A C A

Removing "B" from the bag:
remove("B") returns true
The bag contains 5 string(s), as follows: A D A A C

Removing "A" from the bag:
remove("A") returns true
The bag contains 4 string(s), as follows: C D A A

Removing "C" from the bag:
remove("C") returns true
The bag contains 3 string(s), as follows: A D A

Removing "Z" from the bag:
remove("Z") returns false
The bag contains 3 string(s), as follows: A D A

Clearing the bag:
Testing isEmpty with an empty bag:
isEmpty finds the bag empty: OK.

The bag contains 0 string(s), as follows:
```